

Enunciados individuais — Atividade de Recuperação de Frequência

Orientações gerais para todos os alunos

Todos os alunos deverão desenvolver uma aplicação Flutter com um **CRUD completo**, usando uma **classe de modelo de negócio própria**. Cada aluno terá uma classe diferente, para evitar repetição direta entre os trabalhos.

A aplicação deverá iniciar por uma **tela de listagem**. A partir dessa tela, o usuário deverá conseguir realizar todas as operações do CRUD:

- **Cadastrar** um novo registro;
- **Listar** os registros cadastrados;
- **Visualizar ou acessar os dados de um registro**;
- **Editar** um registro existente;
- **Remover** um registro;
- **Persistir os dados localmente** para que não sejam perdidos ao fechar o aplicativo.

A persistência deverá ser feita com **SharedPreferences**, salvando a lista de objetos em formato JSON. Portanto, cada projeto deverá conter:

- Uma classe de modelo de negócio em arquivo próprio;
- Métodos `toJson` e `fromJson` no modelo;
- Uma classe de repositório local para salvar, carregar e limpar os dados;
- Uma tela inicial de listagem;
- Uma tela ou formulário para cadastro e edição;
- Validação de campos obrigatórios;
- Mensagens de feedback com `SnackBar`;
- Mensagem ao usuário sempre que um item for registrado com sucesso;
- Mensagem ao usuário sempre que um item for alterado com sucesso;
- Mensagem ao usuário sempre que um item for removido com sucesso;
- Confirmação obrigatória antes de remover qualquer registro;
- Confirmação antes de limpar dados importantes.

Requisitos mínimos obrigatórios

Cada aluno deverá entregar uma aplicação Flutter contendo:

1. Pasta `models/`, com a classe de modelo de negócio.
2. Pasta `repositories/`, com a classe de repositório local.
3. Pasta `pages/`, com pelo menos uma tela de listagem e uma tela/formulário de cadastro e edição.
4. CRUD completo funcionando.
5. Persistência local com `SharedPreferences`.
6. Conversão da lista de objetos para JSON.

7. Carregamento automático dos dados ao abrir a tela de listagem.
8. README explicando o tema, a classe de negócio criada e o fluxo do CRUD.
9. Mensagens claras ao usuário após cadastro, alteração e remoção.
10. Confirmação antes de qualquer remoção de registro.

Estrutura mínima sugerida do projeto

```
lib/  
  models/  
    nome_do_modelo.dart  
  repositories/  
    nome_do_repositorio.dart  
  pages/  
    list_page.dart  
    form_page.dart  
  main.dart
```

A nomenclatura dos arquivos deve seguir o padrão `snake_case`, enquanto as classes devem usar `PascalCase`.

Critérios de avaliação

Critério	Pontuação
Classe de modelo própria, coerente e diferente das demais	15 pontos
CRUD completo iniciado por tela de listagem	25 pontos
Persistência local com SharedPreferences e JSON	20 pontos
Separação entre modelo, repositório e telas	15 pontos
Validações, SnackBars, mensagens de sucesso e confirmações antes de remover	15 pontos
README explicativo e evidências de execução	10 pontos
Total	100 pontos

Enunciados individuais por aluno

1. Alan Lino Dos Reis

Classe de modelo: `Book`

Tema: CRUD de livros pessoais.

Desenvolva uma aplicação Flutter para gerenciar uma pequena biblioteca pessoal. A tela inicial deverá listar os livros cadastrados. A partir dessa listagem, o usuário deverá conseguir cadastrar, editar e remover livros.

Cada livro deverá conter, no mínimo, os campos: título, autor e ano de publicação. A lista deverá ser persistida localmente com SharedPreferences em formato JSON. O README deverá explicar como a classe `Book` representa o modelo de negócio e como os dados são salvos e carregados pelo repositório.

2. Alexandre Filipi Lustosa Martins

Classe de modelo: `Task`

Tema: CRUD de tarefas pessoais.

Crie uma aplicação Flutter para controle de tarefas. A tela inicial deverá apresentar a lista de tarefas cadastradas e permitir acessar as ações de cadastro, edição, conclusão e remoção.

Cada tarefa deverá conter, no mínimo, título, descrição e status de conclusão. A aplicação deverá persistir a lista localmente com SharedPreferences e JSON. O README deverá explicar a diferença entre manter as tarefas apenas em memória e persistir os registros no dispositivo.

3. Andrielson Leonardo Teza

Classe de modelo: `Contact`

Tema: CRUD de contatos.

Desenvolva um aplicativo Flutter para cadastro de contatos. A tela inicial deverá listar todos os contatos e permitir cadastrar novo contato, editar contato existente e remover contato.

Cada contato deverá conter nome, telefone e e-mail. Os dados deverão ser persistidos localmente usando SharedPreferences. O README deverá explicar o uso de chaves no SharedPreferences e como a lista de contatos é convertida para JSON.

4. Bruno Luis Da Cruz

Classe de modelo: `Movie`

Tema: CRUD de filmes assistidos.

Crie uma aplicação Flutter para registrar filmes assistidos. A tela inicial deverá listar os filmes cadastrados. A partir dela, o usuário deverá conseguir incluir, editar e remover registros.

Cada filme deverá conter título, diretor e avaliação pessoal. A lista deverá ser salva localmente com SharedPreferences. O README deverá explicar quando o SharedPreferences é adequado para dados simples e por que ele não substitui um banco de dados completo em aplicações maiores.

5. Douglas Antonio Garcia

Classe de modelo: `Product`

Tema: CRUD de produtos de uma lista de compras.

Implemente uma aplicação Flutter para gerenciar produtos de uma lista de compras. A tela inicial deverá exibir os produtos cadastrados e permitir cadastrar, editar e remover cada item.

Cada produto deverá conter nome, quantidade e categoria. A remoção deverá pedir confirmação antes de apagar o registro. O README deverá explicar a diferença entre remover um item da lista visual e remover o dado persistido no armazenamento local.

6. Emmanuely Madeira

Classe de modelo: `Pet`

Tema: CRUD de pets.

Crie uma aplicação Flutter para cadastro de pets. A tela inicial deverá listar os pets cadastrados e permitir realizar as operações de cadastro, edição e remoção.

Cada pet deverá conter nome, espécie e nome do tutor. Os dados deverão ser salvos localmente com SharedPreferences. O README deverá explicar como cada objeto `Pet` é convertido para JSON e depois reconstruído ao carregar a aplicação.

7. Enzo Mazzutti Trevisan

Classe de modelo: `Student`

Tema: CRUD de estudantes.

Desenvolva uma aplicação Flutter para cadastro de estudantes. A tela inicial deverá apresentar a listagem dos estudantes e permitir cadastrar, editar e remover registros.

Cada estudante deverá conter nome, matrícula e curso. A lista deverá ser persistida em SharedPreferences usando JSON. O README deverá explicar como a tela de listagem se relaciona com o formulário de cadastro e edição.

8. Fernando Villar Libano

Classe de modelo: `Course`

Tema: CRUD de cursos.

Implemente uma aplicação Flutter para cadastro de cursos. A tela inicial deverá listar os cursos cadastrados e permitir cadastrar, editar e remover cursos.

Cada curso deverá conter nome, sigla e carga horária. A persistência deverá ser feita localmente com `SharedPreferences`. O README deverá explicar como manter a lista em memória sincronizada com o armazenamento local após cada operação do CRUD.

9. Guilherme Campos Feuser

Classe de modelo: `Appointment`

Tema: CRUD de compromissos.

Crie um aplicativo Flutter para organizar compromissos. A tela inicial deverá listar os compromissos cadastrados e permitir adicionar, editar e remover registros.

Cada compromisso deverá conter título, data e local. A aplicação deverá salvar os dados localmente usando JSON e `SharedPreferences`. O README deverá explicar a função do método `toJson` na conversão do objeto para persistência.

10. Hilario Canci Neto

Classe de modelo: `Recipe`

Tema: CRUD de receitas culinárias.

Desenvolva uma aplicação Flutter para cadastro de receitas. A tela inicial deverá listar as receitas cadastradas. A partir dela, o usuário deverá conseguir cadastrar, editar e remover receitas.

Cada receita deverá conter nome, ingredientes principais e tempo de preparo. Os dados deverão ser persistidos localmente. O README deverá explicar o papel do construtor `fromJson` para reconstruir objetos a partir dos dados salvos.

11. Joao Pedro Domingues

Classe de modelo: `Workout`

Tema: CRUD de treinos físicos.

Crie uma aplicação Flutter para registro de treinos. A tela inicial deverá listar os treinos cadastrados e permitir cadastrar, editar e remover cada treino.

Cada treino deverá conter nome do exercício, número de séries e número de repetições. A lista deverá ser salva em SharedPreferences com JSON. O README deverá explicar o fluxo completo: objeto Dart, Map, String JSON e armazenamento local.

12. Joao Vitor Zilio Silva

Classe de modelo: Expense

Tema: CRUD de despesas pessoais.

Implemente uma aplicação Flutter para controle de despesas. A tela inicial deverá listar as despesas cadastradas e permitir cadastrar, editar e remover registros.

Cada despesa deverá conter descrição, valor e categoria. Os dados deverão ser carregados automaticamente ao abrir a listagem, usando `initState`. O README deverá explicar por que o carregamento inicial dos dados deve ocorrer na criação do estado da tela.

13. Kaique Vinicius Patricio De Sousa

Classe de modelo: Vehicle

Tema: CRUD de veículos.

Desenvolva uma aplicação Flutter para cadastro de veículos. A tela inicial deverá exibir os veículos cadastrados e permitir cadastrar, editar e remover cada registro.

Cada veículo deverá conter modelo, placa e ano. A função de cadastro deverá validar os campos, criar o objeto, adicionar à lista, limpar o formulário e salvar os dados persistidos. O README deverá explicar cada etapa da função de cadastro.

14. Layssa Rodrigues Alves

Classe de modelo: Event

Tema: CRUD de eventos.

Crie uma aplicação Flutter para cadastro de eventos. A tela inicial deverá listar os eventos cadastrados e permitir cadastrar, editar e remover eventos.

Cada evento deverá conter nome, data e local. A remoção de um evento deverá pedir confirmação. O README deverá explicar por que remover apenas da lista em memória não é suficiente quando há persistência local.

15. Luan Cesar Costa De Lima

Classe de modelo: `Game`

Tema: CRUD de jogos.

Implemente uma aplicação Flutter para organizar jogos cadastrados pelo usuário. A tela inicial deverá listar os jogos e permitir cadastrar, editar e remover registros.

Cada jogo deverá conter título, plataforma e gênero. A edição deverá preencher o formulário com os dados atuais do registro selecionado. O README deverá explicar como controlar o modo de edição e como salvar a alteração na lista persistida.

16. Luiz Henrique Altissimo

Classe de modelo: `Note`

Tema: CRUD de anotações.

Desenvolva uma aplicação Flutter para cadastro de anotações. A tela inicial deverá listar as anotações cadastradas e permitir criar, editar e remover notas.

Cada anotação deverá conter título, conteúdo e data de criação. A aplicação deverá usar `TextEditingController` para controlar os campos do formulário. O README deverá explicar como os controllers são usados para capturar, preencher e limpar os campos.

17. Marcos Paulo Miranda Soares

Classe de modelo: `Employee`

Tema: CRUD de funcionários.

Crie uma aplicação Flutter para cadastro de funcionários. A tela inicial deverá listar os funcionários cadastrados e permitir cadastrar, editar e remover registros.

Cada funcionário deverá conter nome, cargo e setor. A classe `Employee` deverá ficar em arquivo próprio dentro da pasta `models`. O README deverá explicar por que separar o modelo da tela melhora a manutenção do projeto.

18. Pedro Gabriel Sepulveda Borgheti

Classe de modelo: `Supplier`

Tema: CRUD de fornecedores.

Implemente uma aplicação Flutter para cadastro de fornecedores. A tela inicial deverá listar os fornecedores cadastrados e permitir cadastrar, editar e remover registros.

Cada fornecedor deverá conter nome, CNPJ fictício e telefone. A aplicação deverá manter o modelo em `models/`, o repositório em `repositories/` e as telas em `pages/`. O README deverá explicar o papel de cada pasta na organização do projeto.

19. Pedro Lucas Reis

Classe de modelo: `TravelDestination`

Tema: CRUD de destinos de viagem.

Desenvolva uma aplicação Flutter para cadastro de destinos de viagem. A tela inicial deverá listar os destinos cadastrados e permitir cadastrar, editar e remover cada destino.

Cada destino deverá conter cidade, país e descrição curta. A tela de listagem deverá importar corretamente o modelo usando caminho relativo. O README deverá explicar como o Dart localiza arquivos em pastas diferentes usando imports relativos.

20. Rodrigo Da Silva Pereira

Classe de modelo: `InventoryItem`

Tema: CRUD de itens de estoque.

Crie uma aplicação Flutter para controle simples de estoque. A tela inicial deverá listar os itens cadastrados e permitir cadastrar, editar e remover registros.

Cada item deverá conter nome, quantidade e localização no estoque. A tela não deverá acessar diretamente o `SharedPreferences`; essa responsabilidade deverá ficar em uma classe de repositório. O README deverá explicar como o repositório encapsula a lógica de persistência.

21. Shara Palharini Lima

Classe de modelo: `Habit`

Tema: CRUD de hábitos.

Implemente uma aplicação Flutter para acompanhamento de hábitos. A tela inicial deverá listar os hábitos cadastrados e permitir cadastrar, editar e remover hábitos.

Cada hábito deverá conter nome, frequência desejada e observação. A aplicação deverá validar campos obrigatórios, exibir mensagens com `SnackBar` e confirmar ações destrutivas. O README deverá explicar por que feedbacks claros melhoram a experiência do usuário e evitam retornos silenciosos.

Observação final

Cada aluno deverá desenvolver uma aplicação própria, com uma classe de negócio diferente, mas todos deverão seguir a mesma arquitetura mínima: modelo, repositório, tela de listagem e formulário de cadastro/edição. A tela de listagem deve ser o ponto de entrada do CRUD, concentrando o acesso às operações principais.